

Módulo 10 – Capítulo 03 – Sección 01

Model registry: catálogo centralizado de modelos, versiones y metadatos

Cuando una organización tiene más de tres modelos en producción, inevitablemente enfrenta una pregunta que parece trivial pero se vuelve paralizante: ¿cuál es exactamente el modelo que está sirviendo este endpoint en producción ahora mismo, y cómo fue creado? Sin un model registry centralizado, esa pregunta no tiene una respuesta confiable. Los pesos del modelo viven en un bucket de S3 con un path que alguien consideró descriptivo hace ocho meses; el código que lo entrenó podría estar en algún commit de un branch que ya fue eliminado; las métricas de evaluación que justificaron su despliegue podrían estar en un notebook de Jupyter en el laptop del ML Engineer que lo creó. El model registry existe para eliminar permanentemente esta clase de entropía.

Un model registry es el sistema de registro centralizado que actúa como source of truth para todos los modelos de una organización, almacenando no solo los artefactos binarios —pesos del modelo, tokenizer, configuración— sino también los metadatos que los hacen utilizables y auditables. Esos metadatos incluyen: versión semántica, framework y versión del framework, hash SHA256 del artefacto para integridad verificable, métricas de evaluación offline, dataset de entrenamiento y su versión exacta, hiperparámetros de entrenamiento, estado del ciclo de vida (Staging/Production/Archived con timestamps de cada transición), y trazabilidad hacia el experimento de MLflow o el pipeline de entrenamiento que generó el modelo. La diferencia entre un object storage con archivos nombrados y un model registry es la diferencia entre una colección de archivos y un sistema gobernado: el registry implementa el patrón de gestión de estados con transiciones controladas que garantizan que ningún modelo llega a producción sin pasar por los gates de aprobación definidos.

La gestión de estados es el mecanismo central del registry. Un modelo no puede transicionar de Staging a Production sin que alguien —o algo— ejecute explícitamente esa transición con una justificación registrada. En MLflow, esto se implementa con `client.transition_model_version_stage("my-model", 1, "Production")`, que registra el cambio de estado en el audit log con el usuario que lo ejecutó y el timestamp. Pero la

transición manual es solo el mecanismo de bajo nivel: las implementaciones maduras añaden gates automáticos antes de permitir la transición —verificación de que las métricas de evaluación superan el umbral definido, test de compatibilidad del modelo con la capa de serving, y aprobación del model card por el equipo de governance. Los webhooks de notificación que se disparan cuando un modelo cambia de estado permiten que los sistemas de CI/CD downstream —el pipeline de despliegue, el sistema de notificación a Slack, el motor de configuración de GitOps— reaccionen automáticamente a las transiciones.

Las implementaciones más adoptadas de model registry tienen posicionamientos distintos que determinan su idoneidad. **MLflow Model Registry** es el estándar open source: flexible, integrable con cualquier cloud y cualquier framework, y con una comunidad amplia respaldada por la Linux Foundation. **SageMaker Model Registry** es la opción managed dentro del ecosistema AWS, con integración nativa con SageMaker Pipelines y menor overhead operativo para organizaciones ya comprometidas con AWS. **Vertex AI Model Registry** es la alternativa equivalente en GCP. **Hugging Face Hub Enterprise** está optimizado para modelos de lenguaje con soporte nativo para los formatos de Hugging Face Transformers. La elección correcta depende del cloud provider principal, del volumen de artefactos, y del grado de integración requerida con el pipeline de CI/CD existente.

Conceptos clave del model registry

- **Artefact storage:** los pesos del modelo se almacenan en object storage (S3, GCS) con URIs inmutables por versión; el registry almacena el puntero y los metadatos, no el binario directamente; la inmutabilidad del URI garantiza reproducibilidad.
- **Estado del ciclo de vida:** transiciones explícitas entre None, Staging, Production y Archived con timestamps, responsable y justificación registrados en el audit log; cada transición es una decisión documentada, no un cambio silencioso.
- **Linaje de modelos:** referencia trazable al experimento (run_id de MLflow), al dataset (URI + versión de DVC o LakeFS), y al código (commit SHA del repositorio de entrenamiento); el triángulo datos-código-modelo completamente trazable.
- **Tags y metadatos personalizados:** campos adicionales específicos de la organización como `compliance_reviewed`, `bias_evaluation`, `serving_latency_p99_ms` y `business_owner`; extienden el registry sin modificar su estructura base.
- **Webhooks de estado:** notificaciones automáticas a Slack, PagerDuty o sistemas de CI/CD cuando un modelo cambia de estado; permiten despliegues automáticos al pasar a Production sin intervención manual del equipo de plataforma.

Nota del Arquitecto: *El model registry no reemplaza el almacenamiento de artefactos: lo complementa añadiendo metadatos, gestión de estados y trazabilidad que hacen que un binario sea un modelo gestionado. La pregunta de diseño más importante al implementar un registry es cuándo automatizar las transiciones de estado y cuándo requerir aprobación humana: la respuesta depende del nivel de riesgo del modelo, no de la comodidad del proceso.*

El model registry es el componente de la plataforma que más impacto tiene sobre el governance: sin él, ninguna política de aprobación o auditoría de modelos puede aplicarse de forma sistemática. La sección siguiente examina MLflow como la implementación más adoptada del model registry, cubriendo su arquitectura interna y las mejores prácticas de integración con pipelines de MLOps.

Módulo 10 – Capítulo 03 – Sección 02

MLflow: experiments, runs, modelos y serving integrado

MLflow ocupa una posición singular en el ecosistema de MLOps: es la plataforma open source más adoptada en la industria para tracking de experimentos y gestión del ciclo de vida de modelos, respaldada por la Linux Foundation y con contribuciones activas de Databricks, Microsoft, NVIDIA y decenas de otras organizaciones. Su diseño unifica en un solo sistema cuatro capacidades que históricamente requerían herramientas separadas: el tracking de experimentos, el empaquetado de modelos, el model registry y el serving básico. Esta integración nativa es su principal ventaja: un modelo entrenado con PyTorch puede ser registrado directamente desde el código de entrenamiento y estar disponible como endpoint REST sin salir del ecosistema de MLflow.

El componente de **MLflow Tracking Server** registra cada ejecución de entrenamiento (run) con un nivel de detalle configurable que incluye parámetros de entrada, métricas iteración por iteración (loss, accuracy, BLEU), artefactos generados (pesos, plots, confusion matrix, eval reports), y el código fuente via git commit SHA. Cada run pertenece a un experiment que agrupa runs relacionados, y puede ser comparado visual o programáticamente: la UI de MLflow permite seleccionar múltiples runs y visualizar sus métricas en paralelo para identificar qué configuraciones producen los mejores resultados, y la API REST permite hacer las mismas comparaciones en notebooks o scripts de análisis automático. El autologging —activado con una sola línea `mlflow.autolog()`— captura automáticamente parámetros y métricas para los frameworks más populares (sklearn, xgboost, lightgbm, pytorch-lightning, transformers de Hugging Face) sin que el ML Engineer tenga que añadir código de logging manualmente.

El **Model Registry** de MLflow añade el gobierno del ciclo de vida sobre los modelos exitosos. Desde un run exitoso, el modelo se registra con `mlflow.register_model(run_id, "my-model")` creando automáticamente la versión 1 en el registry. La gestión de estados se hace con `client.transition_model_version_stage("my-model", 1, "Production")`, que registra el cambio de estado en el audit log con el usuario y el timestamp. La API REST del registry es completamente consumible desde pipelines de CI/CD: un script de GitHub Actions puede

consultar si el modelo en Staging supera las métricas de threshold definidas y, si es así, promoverlo automáticamente a Production usando una llamada a la API REST de MLflow sin intervención humana. Esta integración con CI/CD es lo que transforma el registry de un catálogo manual a un componente activo del pipeline de MLOps.

El componente de **MLflow Models** define un formato de empaquetado estándar con un archivo `MLmodel` YAML que especifica el flavor del modelo (sklearn, pytorch, tensorflow, transformers, pyfunc para modelos custom), las dependencias de Python con sus versiones exactas, y la firma de la API (input/output schema validado con tipos de datos Pandas o Tensor). Este formato estándar es lo que permite que `mlflow models serve -m "models:/my-model/Production" -p 5000` funcione sin código adicional: MLflow sabe cómo cargar el modelo, qué dependencias instalar y qué schema de API exponer. La firma del modelo es especialmente importante para detectar errores de integración: si el sistema consumidor envía un input con el schema incorrecto, el error se detecta en la capa de serving antes de llegar al modelo.

Aspectos técnicos de MLflow

- **MLflow Tracking:** API Python (`mlflow.log_param` , `mlflow.log_metric` , `mlflow.log_artifact`) con autologging automático para sklearn, xgboost, pytorch-lightning y transformers; el servidor centralizado garantiza que ningún run quede solo localmente.
- **MLflow Projects:** especificación de entornos reproducibles via `MLproject` YAML con conda environment o Docker container; `mlflow run git+https://github.com/org/repo.git#path` reproduce cualquier experimento exactamente.
- **MLflow Models:** formato de empaquetado estándar con `MLmodel` YAML; la firma de la API (input/output schema) detecta errores de integración antes de que lleguen al modelo en producción.
- **MLflow Registry REST API:** endpoints para crear modelos, versiones, transiciones de estado y búsqueda, completamente consumibles desde CI/CD pipelines via `curl` o el SDK Python; el registry es programable, no solo una UI.
- **Databricks Managed MLflow:** versión enterprise con RBAC integrado, Unity Catalog para linaje de datos y modelos, workspace isolation, y Feature Store nativo integrado con el Registry; reduce la carga operativa de autogestionar el tracking server.

Nota del Arquitecto: Usar `mlflow.set_tracking_uri()` con el servidor centralizado de la plataforma desde el inicio de cada experimento —idealmente en el código de inicialización del entorno de entrenamiento, no en el código de los ML Engineers— garantiza que ningún run quede registrado solo localmente y se pierda al destruir el entorno de desarrollo. Esta configuración centralizada es una de las pocas que justifica obligar a los equipos a usarla: las consecuencias de omitirla son pérdida irreversible de resultados de experimentos.

MLflow proporciona la base técnica sobre la que los demás componentes del model registry pueden construirse: el tracking de experimentos que captura el linaje de cada modelo, el registry que gestiona su ciclo de vida, y el serving que lo expone como API. La siguiente sección examina Hugging Face Hub Enterprise como alternativa especializada para organizaciones cuya infraestructura de modelos está centrada en LLMs y el ecosistema de Hugging Face Transformers.

Módulo 10 – Capítulo 03 – Sección 03

Hugging Face Hub Enterprise: gestión de modelos privados y control de acceso

Hugging Face Hub ha transformado la forma en que los equipos de IA acceden, comparten y despliegan modelos de lenguaje: con más de 500,000 modelos públicos disponibles via `from_pretrained()` y un ecosistema de herramientas (transformers, datasets, evaluate, peft) que cubre el pipeline completo de desarrollo de LLMs, se ha convertido en la infraestructura de facto del ecosistema de modelos de lenguaje abiertos. Sin embargo, las organizaciones que desarrollan modelos propietarios enfrentan un problema fundamental: necesitan beneficiarse del mismo ecosistema de herramientas y la misma experiencia de usuario del Hub público, pero sin exponer sus modelos, datasets y aplicaciones internos al mundo externo. Hugging Face Hub Enterprise —también llamado Hugging Face Enterprise Hub— resuelve exactamente este problema.

Hugging Face Hub Enterprise es la versión privada y gestionada del Hub público, diseñada para alojar modelos propietarios, datasets privados y Spaces internos con los mismos mecanismos de acceso que el Hub público (`huggingface-cli` , `from_pretrained()` , Inference Endpoints) pero con controles de acceso corporativos. La primera diferencia operacional respecto al Hub público es la **integración con el IdP corporativo**: SSO via SAML 2.0 o OIDC conecta el Hub Enterprise con Okta, Azure AD o Google Workspace, de modo que los empleados acceden con sus credenciales corporativas existentes sin necesitar gestionar una contraseña adicional, y el aprovisionamiento y revocación de acceso se produce automáticamente cuando alguien entra o sale de la organización.

El **modelo de control de acceso** se estructura en tres niveles jerárquicos. A nivel de organización, el administrador gestiona el billing, la configuración de SSO y las políticas de acceso globales. A nivel de equipo, los grupos de usuarios heredan permisos que el administrador de la organización puede configurar con granularidad de lectura, escritura o administración por repositorio. A nivel de repositorio individual, los modelos pueden ser `private` (visible solo al owner), `internal` (visible a todos los miembros de la organización) o `gated` (requiere aprobación explícita para descarga). Esta estructura permite, por ejemplo,

que el modelo de detección de fraude desarrollado por el equipo de riesgo sea `internal` para todos los empleados de la organización pero `gated` para equipos externos a riesgo que quieren usarlo en producción, creando un proceso de aprobación controlado sin la rigidez de un sistema de tickets manual.

Los **Inference Endpoints Enterprise** son la capacidad de despliegue que diferencia Hugging Face Hub Enterprise de un simple repositorio de modelos. Permiten desplegar modelos privados como APIs en AWS, GCP o Azure con autoscaling automático, VPC peering para que el tráfico no salga a internet, y soporte para GPUs (A10G, A100, L4) sin que el equipo necesite gestionar la infraestructura de Kubernetes subyacente. El modelo de pricing es por hora de endpoint activo, similar a los servicios de inferencia managed de los cloud providers, con la ventaja de que el mismo mecanismo de control de acceso del Hub gestiona quién puede llamar al endpoint. Para organizaciones con requisitos de residencia de datos, el Hub Enterprise puede desplegarse en infraestructura VPC del cliente (AWS, GCP, Azure) con los datos y modelos permaneciendo dentro del perímetro de la organización.

Componentes principales de Hugging Face Hub Enterprise

- **Private model repositories:** repositorios Git LFS para modelos con acceso restringido a teams específicos, con política de descarga controlada via tokens de usuario con scopes limitados (`read` , `write` , `admin`); el mismo CLI que los desarrolladores ya conocen del Hub público.
- **Inference Endpoints Enterprise:** despliegue de modelos privados como APIs en AWS/GCP/Azure con autoscaling, VPC peering y sin salida de datos a internet; soporte para GPUs A10G, A100 y L4; facturación por hora de uso.
- **SSO y RBAC:** integración con SAML/OIDC para autenticación federada; permisos granulares por repositorio con roles predefinidos (reader, contributor, admin); sincronización automática con grupos del IdP corporativo.
- **Audit logs:** registro inmutable de todas las acciones (push, pull, delete, permission change) con timestamp, usuario e IP, exportable a SIEM corporativo via webhook o API; cumple con requisitos de auditoría de GDPR y SOC 2.
- **Spaces privados:** aplicaciones Gradio y Streamlit internas para demos y herramientas de evaluación de modelos, accesibles solo desde la red corporativa; permite construir herramientas de evaluación interna con la misma tecnología del Hub público.

Nota del Arquitecto: *Hugging Face Hub Enterprise es especialmente valiosa en organizaciones donde los ML Engineers ya usan activamente el ecosistema público de Hugging Face: la curva de aprendizaje de la versión Enterprise es prácticamente cero porque las mismas herramientas (transformers, datasets, huggingface_hub) funcionan de forma idéntica apuntando al servidor privado en lugar del público. El `HF_ENDPOINT` environment variable es lo único que cambia; el resto del código es idéntico.*

Hugging Face Hub Enterprise permite a las organizaciones beneficiarse del ecosistema open source de Hugging Face manteniendo el control total sobre quién accede a sus modelos propietarios y desde dónde. La siguiente sección aborda un problema práctico que emerge cuando los modelos son un producto compartido en la organización: cómo versionar los modelos de forma que los equipos consumidores puedan entender qué cambios implica cada actualización y gestionar sus migraciones de forma planificada.

Módulo 10 – Capítulo 03 – Sección 04

Versionado semántico de modelos: cuándo es un patch, un minor o un major release

El versionado semántico (SemVer: MAJOR.MINOR.PATCH) fue diseñado para software con contratos de API bien definidos, donde la ruptura de backward compatibility es objetivamente determinable. Aplicarlo a modelos de ML requiere adaptar sus convenciones a las características específicas de los modelos: a diferencia de una librería de software donde el contrato es la API pública, en un modelo el contrato incluye simultáneamente la arquitectura, el input schema, el output schema, y —más sutilmente— las distribuciones de predicción esperadas. Un cambio que no toca ningún código puede producir un cambio de distribución de outputs que rompe los sistemas downstream que calibraron sus umbrales de decisión sobre la distribución del modelo anterior. Esta complejidad hace que el versionado de modelos requiera criterios adicionales a los del software convencional.

Un cambio de **PATCH** (1.0.0 → 1.0.1) corresponde a ajustes que no cambian el comportamiento observable del modelo para los sistemas que lo consumen. El caso más común es el reentrenamiento incremental con datos adicionales que mejora las métricas sin cambiar la arquitectura: el modelo procesa los mismos inputs y produce outputs con el mismo schema y una distribución estadísticamente equivalente. Otros casos de PATCH incluyen la corrección de un bug en el pre/postprocessing que no afecta al comportamiento del modelo en inputs correctos, o la actualización de dependencias de serving (versión de TorchServe, actualización de la imagen base) sin cambiar el modelo en sí. La regla práctica es: si los sistemas downstream no necesitan cambiar su código ni recalibrar sus umbrales, es un PATCH.

Un cambio de **MINOR** (1.0.0 → 1.1.0) corresponde a mejoras backward-compatible: el modelo gana capacidades nuevas o mejora su calidad en áreas específicas, pero los sistemas que lo consumen con el input existente siguen recibiendo outputs válidos con el mismo schema. El fine-tuning con una nueva tarea adicional que mantiene el rendimiento en las tareas existentes es un MINOR: el modelo ahora puede hacer más, pero no hace menos. La mejora de latencia por quantización (INT8 o INT4) sin degradación de calidad superior a la

tolerancia definida también es un MINOR: el modelo es más eficiente, pero su comportamiento desde la perspectiva del consumidor es equivalente. El criterio es: si los sistemas downstream pueden actualizar al nuevo modelo sin cambiar código ni recalibrar umbrales, es un MINOR.

Un cambio de **MAJOR** (1.0.0 → 2.0.0) indica que algo en el contrato del modelo ha cambiado de forma que los sistemas downstream necesitan adaptarse. Los casos más claros son el cambio de arquitectura base (pasar de BERT a RoBERTa o de GPT-2 a Mistral), el cambio en el input schema (añadir un campo obligatorio nuevo o cambiar el tipo de un campo existente), o el cambio en el output schema (cambiar la estructura de la respuesta o los nombres de los campos). Más difícil de determinar pero igualmente importante es el cambio en la distribución de outputs que requiere recalibración de los sistemas downstream: si el modelo anterior asignaba probabilidades de 0.9+ a los casos de alta confianza y el nuevo modelo distribuye la misma confianza entre 0.7-0.85, los sistemas que tenían umbrales basados en la distribución anterior necesitan ser recalibrados. Sin comunicación explícita y período de transición, un cambio MAJOR silencioso en un modelo de producción puede degradar silenciosamente la calidad de múltiples sistemas downstream.

Puntos críticos del versionado semántico de modelos

- **PATCH release:** reentrenamiento incremental (misma arquitectura, mismos hiperparámetros, datos adicionales), bugfix en pre/postprocessing, actualización de dependencias de runtime sin cambio de modelo; los consumidores no necesitan cambiar nada.
- **MINOR release:** fine-tuning que añade capacidades nuevas sin romper las existentes, mejora de latencia por optimización (quantización, pruning, distilación) con degradación de calidad dentro de tolerancia definida; los consumidores se benefician automáticamente.
- **MAJOR release:** cambio de arquitectura base, cambio en el input schema (añadir campo obligatorio), cambio en el output schema, cambio en el idioma/dominio que invalida evaluaciones previas; requiere proceso formal de migración.
- **Comunicación de breaking changes:** notificación con mínimo 30 días de antelación a los equipos consumidores, con guía de migración específica (qué cambió, cómo actualizar el código, cómo recalibrar los umbrales) y período de soporte del modelo anterior en paralelo durante la transición.
- **Automated compatibility testing:** test suite que verifica que el nuevo modelo (en Staging) produce outputs dentro de umbrales aceptables comparado con el modelo en

Production antes de autorizar la promotion; incluye test de distribución de predicciones además de métricas de calidad.

Nota del Arquitecto: *El error más frecuente en versionado de modelos es subestimar el impacto de los cambios de distribución de outputs. Un modelo nuevo puede superar al anterior en todas las métricas de evaluación offline y aun así romper sistemas downstream que calibraron sus reglas de negocio sobre la distribución del modelo anterior. La regla práctica es: antes de cualquier cambio MAJOR, pedir a los equipos consumidores que evalúen el impacto sobre sus umbrales de decisión con datos de producción, no solo con el test set del equipo que desarrolla el modelo.*

La versión de un modelo es un contrato con los sistemas que lo consumen: incrementar el MAJOR sin notificar y proveer guía de migración es equivalente a introducir un breaking change silencioso en una librería de software con múltiples dependencias. La sección siguiente aborda el problema relacionado del linaje de modelos: cómo rastrear la historia completa de cómo se llegó a cada versión del modelo, desde los datos de entrenamiento originales hasta el endpoint que la sirve.

Módulo 10 – Capítulo 03 – Sección 05

Linaje de modelos: trazabilidad desde los datos de entrenamiento hasta el despliegue

El linaje de modelos es la capacidad de reconstruir la historia completa de cómo se llegó a un modelo en producción: qué datos exactos se usaron (con sus versiones y hashes), qué código y configuración se ejecutó (con commit SHA verificable), qué métricas de evaluación se obtuvieron, quién lo aprobó para producción, y cuándo y cómo fue desplegado. Esta trazabilidad completa no es una preocupación académica: es el requerimiento operacional mínimo para tres escenarios que ocurren regularmente en organizaciones que operan IA en producción.

El primer escenario es la **auditoría de compliance**. Cuando un regulador, un auditor externo o el equipo legal pregunta "¿qué datos de clientes europeos fueron usados en el modelo de scoring de crédito que desplegaron el 15 de marzo?", la respuesta debe ser recuperable en minutos con evidencia verificable, no en días con investigación manual. El GDPR, el EU AI Act y regulaciones sectoriales como FINRA (para modelos financieros) o HIPAA (para modelos médicos) establecen requisitos explícitos de documentación que el linaje de modelos satisface técnicamente: cada dataset de entrenamiento tiene un hash verificable que permite determinar exactamente qué registros fueron incluidos.

El segundo escenario es la **investigación de incidentes**. Cuando un modelo de producción produce predicciones incorrectas que afectan a usuarios, la investigación forense necesita responder si el problema es el modelo actual, si era igual de incorrecto en versiones anteriores (regresión histórica), o si el comportamiento correcto es reconstruible con los datos exactos del incidente. Sin linaje completo, esta investigación requiere semanas de búsqueda en logs, comunicación con múltiples equipos y reconstrucción parcial basada en memoria. Con linaje completo, la respuesta está en el model registry: qué experimento generó el modelo, con qué dataset, con qué código, en qué fecha.

El tercer escenario es la **reproducibilidad de experimentos**. En un campo donde el progreso se mide por la capacidad de mejorar sobre resultados anteriores, la reproducibilidad

exacta no es una práctica de calidad opcional: es la base de la metodología científica aplicada a ML. Un equipo que quiere comparar un nuevo enfoque contra el resultado de un experimento de hace seis meses debe poder reproducir exactamente ese experimento — mismo dataset, mismo código, mismo entorno— o la comparación no es válida. El linaje de modelos, al registrar automáticamente el hash del dataset y el commit SHA del código en cada run, hace que esa reproducibilidad sea posible sin depender de la memoria del ingeniero que lo ejecutó.

La implementación técnica del linaje conecta cuatro sistemas con punteros explícitos entre ellos. El sistema de **versionado de datos** (DVC o LakeFS) provee el identificador único del dataset: el hash MD5 en DVC o el commit SHA del data lake en LakeFS, ambos registrables como metadato del run en MLflow. El **experiment tracker** (MLflow) provee el `run_id` que conecta el código (git commit SHA), los hiperparámetros, las métricas y los artefactos de salida en un registro inmutable. El **model registry** agrega el estado del ciclo de vida y los metadatos de aprobación sobre los artefactos del experiment tracker. El **sistema de despliegue** (Kubernetes con ArgoCD) registra cuándo, con qué configuración y a qué cluster fue desplegado el modelo. Herramientas como OpenLineage y Marquez implementan el estándar de linaje abierto que permite conectar estos sistemas de forma agnóstica al orquestador específico.

Aspectos técnicos del linaje de modelos

- **Dataset lineage:** cada entrenamiento registra el URI de los datos con versión explícita (`s3://data-lake/features/user_events/v2.3.1@sha256:abc123`), no solo la ruta del directorio; el hash garantiza que la versión referenciada es exactamente la que se usó.
- **Code lineage:** referencia al commit SHA del repositorio de entrenamiento, la imagen Docker con su digest (no solo el tag, que es mutable), y el hash del archivo de configuración de hiperparámetros.
- **Experiment lineage:** el `run_id` de MLflow, el `experiment_id`, y los links a los artefactos intermedios (checkpoints, plots de training curves, eval reports) que permiten reconstruir el proceso de entrenamiento.
- **Deployment lineage:** registro en el model registry de cuándo, quién y con qué configuración de Kubernetes fue desplegado el modelo, incluyendo el ArgoCD Application y la versión del Helm chart utilizada.
- **Downstream impact tracking:** registro de qué servicios consumen cada versión de cada modelo, permitiendo evaluar el impacto antes de una actualización y notificar automáticamente a los equipos afectados antes de un cambio MAJOR.

Nota del Arquitecto: *El linaje de modelos debe ser automático, no manual. Si un ingeniero necesita recordar completar los campos de linaje en el registry, inevitablemente se omitirán en los momentos de mayor presión operativa — precisamente cuando más se necesitan. La implementación correcta es instrumentar el pipeline de entrenamiento para que capture y registre el linaje automáticamente en cada run, con los ingenieros ni siquiera conscientes de que está ocurriendo.*

El linaje de modelos completo y automático es la diferencia entre un modelo en producción que puede ser auditado, reproducido y mantenido con confianza, y uno que existe en un estado de opacidad que solo puede abordarse si quien lo creó sigue en la organización y recuerda los detalles. La sección de cierre de este capítulo sintetiza por qué el model registry, con todas sus capacidades de gestión de estados, linaje y governance integrado, es el componente de la plataforma que hace posible que el gobierno técnico de los modelos escale más allá del esfuerzo manual.

Módulo 10 – Capítulo 03 – Sección 06

Cierre: el model registry es el source of truth de todos los modelos de la organización

Sin un model registry centralizado, los modelos de una organización existen en un estado de entropía que se vuelve insostenible a medida que el número de modelos y equipos crece. Los síntomas de esta entropía son reconocibles: versiones del modelo nombradas con sufijos como `model_final_v3_REAL.pkl` dispersas en buckets de S3 sin estructura; endpoints de producción cuyo modelo subyacente nadie puede identificar sin investigación manual; equipos que descubren a través de un incidente de producción que el modelo que creían estar usando fue reemplazado silenciosamente hace tres semanas. Este estado no es el resultado de ingenieros descuidados: es el resultado esperado de operar sin la infraestructura correcta.

El model registry transforma este caos en un sistema gobernado donde cada modelo tiene una identidad única, un historial de versiones trazable, un estado explícito que refleja su posición en el ciclo de vida, y metadatos suficientes para que cualquier ingeniero —incluyendo alguien que no estuvo presente cuando el modelo fue creado— pueda entender qué hace el modelo, cómo fue creado, quién lo aprobó y quién es responsable de él. Esta transparencia no es solo una ventaja operacional: es un requisito regulatorio para categorías crecientes de sistemas de IA bajo el EU AI Act y las regulaciones sectoriales que lo complementan.

La implementación efectiva de un model registry requiere integración bidireccional con el pipeline de CI/CD. En la dirección de entrenamiento hacia el registry: los pipelines de entrenamiento registran modelos automáticamente en MLflow con todos los metadatos de linaje al completar exitosamente, sin que el ML Engineer tenga que ejecutar ningún paso adicional. En la dirección del registry hacia el sistema de despliegue: los cambios de estado en el registry disparan automáticamente los flujos de despliegue o notificación correspondientes via webhooks — cuando un modelo pasa a Production, ArgoCD actualiza la configuración del InferenceService de KServe sin intervención manual. Esta automatización bidireccional es lo que convierte el registry de un catálogo pasivo en un motor activo del pipeline de MLOps.

En organizaciones con decenas de modelos en producción, el model registry también actúa como **inventario estratégico** de la cartera de modelos. Permite identificar qué modelos están usando datasets que se van a deprecar (impacto cross-cutting que sin el registry requiere investigación manual en múltiples equipos), qué modelos tienen la mayor frecuencia de actualización y por tanto requieren mayor inversión en automatización de su pipeline de reentrenamiento, qué modelos llevan más de seis meses sin actualizarse y podrían estar acumulando concept drift, y qué modelos consumen los recursos de inferencia más caros y son por tanto los candidatos prioritarios de optimización para el equipo de FinOps. El registry como inventario estratégico convierte la información técnica sobre los modelos en insight de gestión para priorizar la inversión del equipo de plataforma.

Principio rector

Un modelo sin registro es un artefacto sin identidad: no puede ser auditado, no puede ser reproducido, y no puede ser mantenido con confianza en producción. El model registry no es un añadido opcional al stack de MLOps: es la infraestructura base sobre la que el governance, el compliance y la reproducibilidad se construyen.

"Without data, you're just another person with an opinion."

— W. Edwards Deming, estadístico y pionero de la gestión de calidad, cuya filosofía de mejora continua basada en datos es el fundamento del MLOps moderno.